WEEK 1 · LESSON 1 OF 24 · HANDOUT

What Machine Learning Really Is — and scikit-learn

Study time: **about 45 minutes** (reading, video and activity)

▶ **Watch:** https://www.youtube.com/watch?v=Gv9_4yMHFhI — open the lesson online to watch inside the course.

🎓 **Go deeper:** Data School: What is machine learning, and how does it work? — https://www.youtube.com/watch?v=elojMnjn4kk

🎓 **Go deeper:** Stanford Machine Learning, Lecture 1 — Andrew Ng — https://www.youtube.com/watch?v=UzxYlbK2c7E

🎓 **Go deeper:** freeCodeCamp: scikit-learn — full course — https://www.youtube.com/watch?v=pqNCD_5r0IU

Your goal in this lesson: Explain what a machine-learning model is, name the steps every scikit-learn project follows, and train and test your first model in Google Colab.

From rules to learning

In ordinary programming, **you** write the rules: "if the house has 3 bedrooms, the rent is 300,000 RWF". Real life is too messy for that. Rent depends on size, district, distance to town, furniture — and on things nobody wrote down.

In **machine learning** you give the computer many **examples** (homes and their real rents), and it **learns the rule by itself**. The learned rule is called a **model**. Then you use the model on new homes it has never seen.

> *Data + an algorithm → a trained model → predictions on new data.*

This course is the hands-on part of the WAMUGII AI Program: you will build the most important kinds of models used in industry, and learn how to check them honestly. You need the Python and Pandas from **Math & Python for AI**.

The two big kinds of supervised learning

Problem	What you predict	Example from this course
Regression	A number	The monthly rent of a home in Kigali
Classification	A category (yes/no, A/B/C)	Will a mobile-money borrower repay the loan?

Both are **supervised** learning: every training example has the correct answer (the **label** or **target**). In Week 4 you will also meet **unsupervised** learning, where there is no answer column.

Meet scikit-learn

scikit-learn (imported as `sklearn`) is the most used Python library for classic machine learning. It is free, already installed in Google Colab, and every model in it works the **same way**:

1. **Choose** a model: `model = LinearRegression()`

2. **Fit** (train) it on training data: `model.fit(X_train, y_train)`
3. **Predict** on new data: `model.predict(X_test)`
4. **Score** it: compare predictions with the real answers.

Learn these four steps once and you can use dozens of algorithms. This is why professionals love scikit-learn.

X and y

scikit-learn expects two things:

- **X** — the **features**: a table with one row per example and one column per input (size, distance…). Written with a capital X because it is a table (a matrix).
- **y** — the **target**: one value per row, the answer to learn (the rent).

Our data: Kigali home rents

Open a new notebook at colab.research.google.com and run this cell. It creates 600 homes. The data is **simulated** (made with code), so everyone gets the same numbers.

```
import numpy as np
import pandas as pd

rng = np.random.default_rng(7)
n = 600
district = rng.choice(["Gasabo", "Kicukiro", "Nyarugenge"], n, p=[0.45, 0.35, 0.2])
bedrooms = rng.integers(1, 5, n)
size_m2 = (bedrooms * 28 + rng.normal(20, 12, n)).clip(25).round()
km_to_centre = rng.uniform(0.5, 15, n).round(1)
furnished = rng.random(n) < 0.3
base = {"Gasabo": 120_000, "Kicukiro": 90_000, "Nyarugenge": 100_000}
rent = (np.array([base[d] for d in district]) + 2_600 * size_m2 - 6_000 * km_to_centre
        + np.where(furnished, 60_000, 0) + rng.normal(0, 25_000, n)).round(-3)
homes = pd.DataFrame({"district": district, "bedrooms": bedrooms, "size_m2": size_m2,
                      "km_to_centre": km_to_centre, "furnished": furnished, "rent_rwf": rent})
print(homes.shape)
homes.head()
```

`homes.head()` shows the first five homes. The rent depends on size, district, distance to the city centre and whether the home is furnished — plus some random noise, like in real life.

Your first model

Start simple: predict rent from **size only**.

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error

X = homes[["size_m2"]]          # double brackets: X must be a table
y = homes["rent_rwf"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

model = LinearRegression()      # 1. choose
model.fit(X_train, y_train)     # 2. fit
pred = model.predict(X_test)    # 3. predict
print("MAE:", round(mean_absolute_error(y_test, pred)), "RWF")   # 4. score
```

You should see an MAE of about **42,000 RWF**: on homes the model never saw, its guesses are off by about 42,000 RWF on average.

Why split the data?

`train_test_split` keeps **20%** of the homes aside as a **test set**. The model learns only from the other 80%. Testing on homes it has never seen tells you how it will do in real life. Testing on the training homes would be like giving students the exam questions the night before — the score would look great and mean nothing.

`random_state=42` makes the random split repeatable, so your numbers match these notes.

Use the model

```
new_home = pd.DataFrame({"size_m2": [100]})
print(model.predict(new_home).round(-3))
print("RWF per extra m²:", model.coef_.round(0))
```

The model predicts about **337,000 RWF** for a 100 m² home and learned that each extra square metre adds about **2,650 RWF** of rent.

What a model is not

- A model is **not** magic: it only finds patterns that are in the data.
- It can be **confidently wrong** for unusual cases (a 400 m² villa — there are none in the data).
- It copies the **past**. If the past was unfair, the model can be unfair too (Week 4).

Key terms

- **Model** — the rule learned from examples.
- **Features (X)** — the inputs; **target (y)** — the answer to predict.
- **Fit** — train the model on data; **predict** — use it on new data.
- **Training set / test set** — data to learn from / data kept aside to check honestly.
- **MAE** — mean absolute error: the average size of the mistakes, in real units.

Remember this: Every scikit-learn project follows the same four steps — choose, fit, predict, score — and is always tested on data the model has never seen.

Name: _____ Date: _____

Activity

Task 1

Change `test_size` to 0.3. Does the MAE change much?

Task 2

Train a model that uses `km_to_centre` only. Is it better or worse than size? Why might that be?

Task 3

Predict the rent of a 60 m² home and a 150 m² home.

Task 4

In your own words, write two sentences explaining to a friend the difference between regression and classification.

Self-check

Think about your answers here, then take the quiz online — it is marked instantly and explains every answer.

1. What is a machine-learning model?
   - A. A rule the computer learned from examples
   - B. A rule written by hand
   - C. A chart
   - D. A database of homes
2. Predicting the monthly rent of a home is which kind of problem?
   - A. Clustering
   - B. Regression
   - C. Classification
   - D. Search
3. What are the four steps every scikit-learn model follows?
   - A. Draw, colour, save, share
   - B. Choose, fit, predict, score
   - C. Open, read, close, print
   - D. Sort, filter, group, plot
4. Why do we keep a test set aside?
   - A. To make training faster
   - B. To make the MAE larger
   - C. Because scikit-learn needs exactly 20%
   - D. To check the model honestly on data it has never seen
5. In scikit-learn, what is y?
   - A. The target: the answer to learn
   - B. The random seed
   - C. The table of features
   - D. The test score
6. The test MAE is 42,000 RWF. What does that mean?
   - A. The rent is 42,000 RWF
   - B. Guesses are off by about 42,000 RWF on average
   - C. The model is 42% accurate
   - D. 42,000 homes were wrong

© WAMUGII TECH SOLUTIONS · wamugii.kento.rw · Original course material — for enrolled students.